



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Olivia Wijaya
NPM	2428240099
Kelas	SI5B
Topik Tugas 1	Toko Laptop / Penjualan Laptop
Entitas utama	laptop
Tautan repositori	(https://github.com/oliviaolip/si5b_kuis1_oliviawijaya.git)
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	8 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Laptop
Entitas utama (tunggal)	laptop
Alamat dasar	/laptops
Field data	id, merk, harga
Field wajib saat POST	merk, harga
Query string untuk filter	merk
Data awal	[{ id: 1, merk: "Asus", harga: 10000000 }, { id: 2, merk: "Lenovo", harga: 12000000 }]
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/laptops	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/laptops/:id	Tidak	200	Ambil satu data
3	POST	/laptops	Ya	201	Tambah data
4	PUT	/laptops/:id	Ya	200	Ubah data
5	DELETE	/laptops/:id	Ya	204	Hapus data

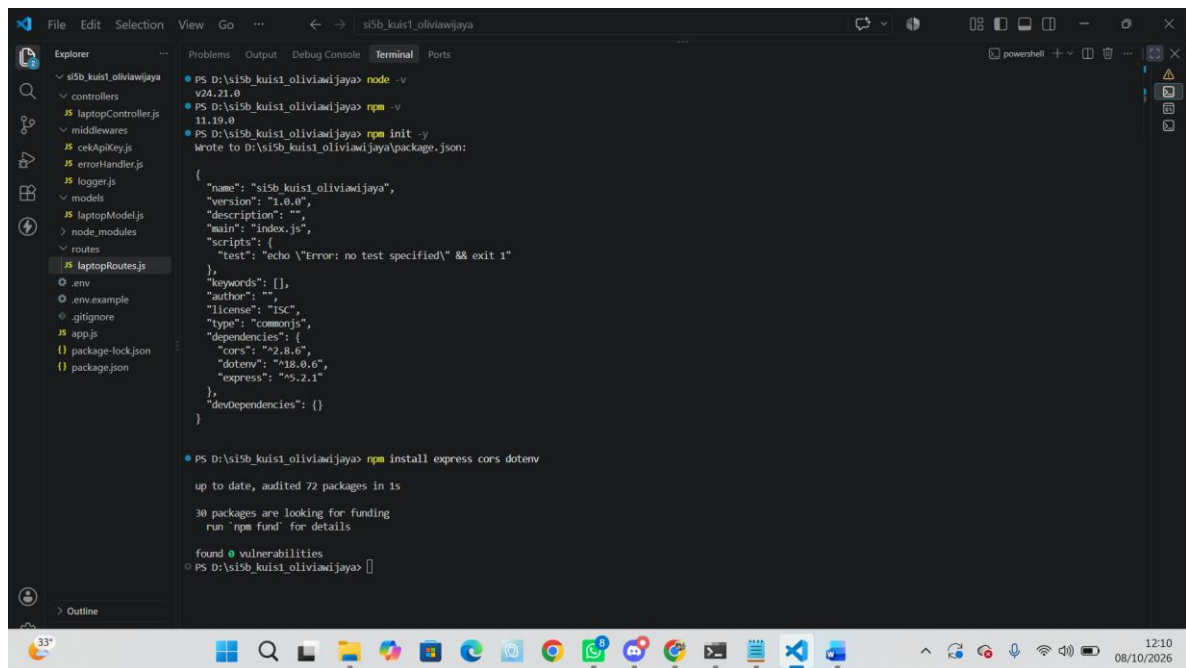
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



```

PS D:\sisb_kuis1_oliviawijaya> node -v
v24.21.0
PS D:\sisb_kuis1_oliviawijaya> npm -v
11.19.0
PS D:\sisb_kuis1_oliviawijaya> npm init -y
wrote to D:\sisb_kuis1_oliviawijaya\package.json:
{
  "name": "sisb_kuis1_oliviawijaya",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^18.0.6",
    "express": "^5.2.1"
  },
  "devDependencies": {}
}

PS D:\sisb_kuis1_oliviawijaya> npm install express cors dotenv
up to date, audited 72 packages in 1s
30 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
PS D:\sisb_kuis1_oliviawijaya>
  
```

[Tempel screenshot di sini]

Gambar 1. Versi Node.js dan instalasi paket

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Jelaskan fungsi *express*, *cors*, dan *dotenv*, serta mengapa versi *Node.js* perlu memenuhi syarat *Express 5*.

- express:** Framework web utama yang digunakan untuk membangun RESTful API, menangani sistem rute (routing), middleware, serta memproses request dan response HTTP.
- cors:** Middleware untuk mengelola kebijakan Cross-Origin Resource Sharing, yang mengizinkan frontend dari domain atau port berbeda untuk mengakses API ini secara aman.
- dotenv:** Pustaka untuk memuat variabel lingkungan dari berkas `.env` ke dalam `process.env`, sehingga konfigurasi sensitif (seperti port dan API key) tersimpan aman di luar kode program.

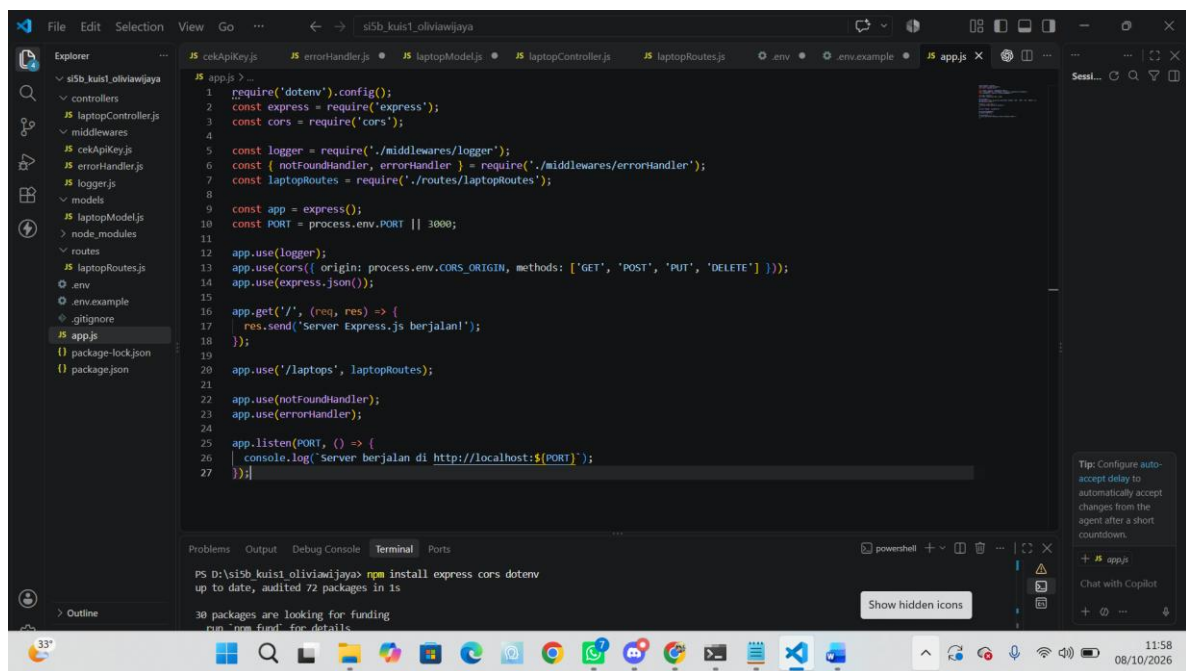
Express 5 memerlukan versi *Node.js* modern (minimal *Node.js* 18 ke atas) karena Express 5 telah memperbarui arsitektur internalnya, termasuk penanganan error otomatis pada fungsi asynchronous (`async/await` dan `Promise`), optimalisasi performa, serta penghentian dukungan untuk metode *Node.js* versi lama (deprecated). Memakai versi *Node.js* yang sesuai memastikan aplikasi berjalan stabil tanpa kendala kompatibilitas.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



[Tempel screenshot di sini]

Gambar 2. Struktur folder project

Gambar 2. Struktur folder project

Penjelasan:

Jelaskan tugas setiap folder dalam satu kalimat.

- controllers: Mengelola logika penanganan request dan response HTTP serta menghubungkan rute dengan pemrosesan data di model.
- middlewares: Menjalankan fungsi perantara untuk memvalidasi, mencatat, atau mengamankan request sebelum mencapai controller.
- models: Mengelola struktur data beserta fungsi manipulasi data aplikasi secara terpisah dari protokol HTTP.
- routes: Memetakan alamat endpoint URL dan metode HTTP ke fungsi controller yang sesuai.
- node_modules: Menyimpan seluruh pustaka atau dependency pihak ketiga yang digunakan untuk menjalankan proyek.

2.2 Ringkasan tugas setiap berkas

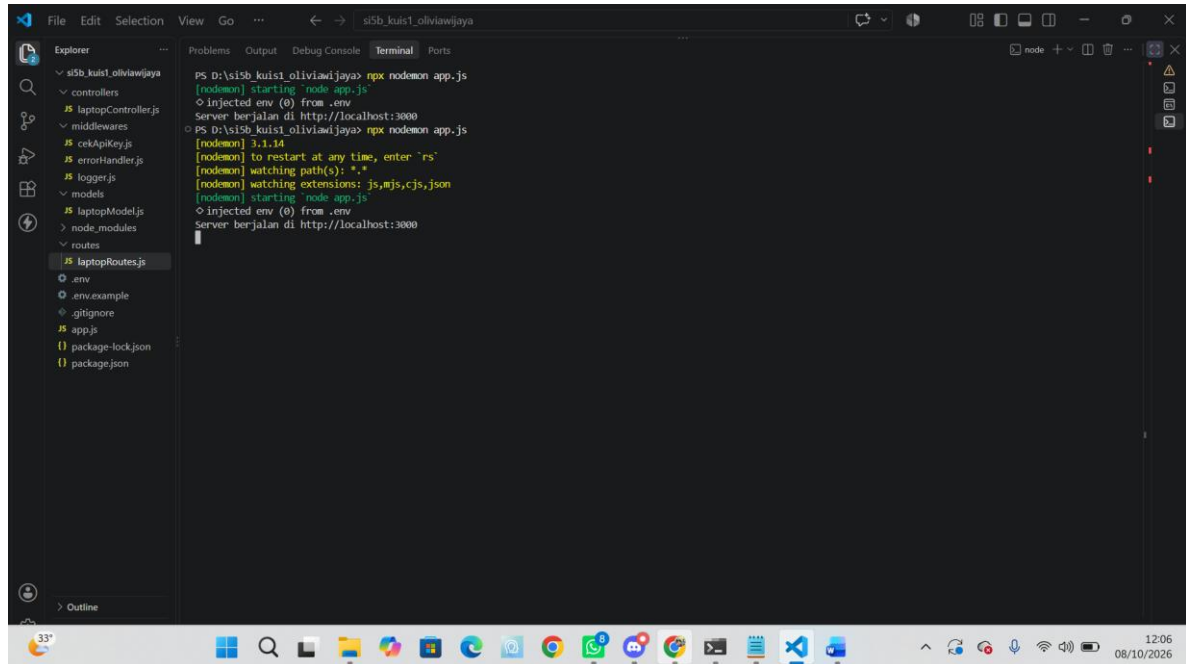
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/laptopModel.js	Mengelola array dataLaptop sebagai penyimpanan data laptop sementara di memori serta menyediakan fungsi manipulasi data seperti getAll, getByld, create, update, dan delete.
controllers/laptopController.js	Menangani logika bisnis untuk setiap endpoint laptop, seperti memvalidasi input merk dan harga, memanggil fungsi di model, serta merespon klien dengan status code dan format JSON yang sesuai.
routes/laptopRoutes.js	Mendefinisikan rute URL untuk resource /laptops, mengarahkan setiap metode HTTP (GET, POST, PUT, DELETE) ke handler pada laptopController, dan menyisipkan middleware cekApiKey pada rute yang membutuhkan proteksi.
middlewares/logger.js	Mengintersepsi setiap request HTTP yang masuk untuk mencatat <i>timestamp</i> , <i>HTTP method</i> , dan <i>URL path</i> ke terminal console sebagai bentuk catatan log server.
middlewares/cekApiKey.js	Memeriksa ketersediaan dan keabsahan nilai header x-api-key dari request terhadap process.env.API_KEY sebelum mengizinkan akses ke controller penambahan, pengubahan, atau penghapusan data.
middlewares/errorHandler.js	Menangani error secara terpusat di seluruh aplikasi, seperti menangani format JSON yang rusak (<i>syntax error</i>) atau error internal server agar selalu menghasilkan respons JSON yang terstruktur.
app.js	Merupakan entry point utama aplikasi yang menginisialisasi Express, memuat middleware global (logger, cors, express.json), menyambungkan rute utama /laptops, serta mengaktifkan server pada port yang ditentukan.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



[Tempel screenshot di sini]

Gambar 3. Server berhasil dijalankan

Gambar 3. Server berhasil dijalankan

Penjelasan:

Jelaskan dari mana nomor port berasal dan apa yang terjadi bila port sudah dipakai aplikasi lain.

Asal Nomor Port

- Nomor port berasal dari variabel lingkungan (environment variable) PORT yang diatur di dalam berkas `.env` (contoh: `PORT=3000`).
- Pada kode `app.js`, nilai tersebut dibaca menggunakan `process.env.PORT`. Jika variabel di `.env` tidak ditemukan, aplikasi akan memakai nilai cadangan (fallback) berupa 3000 melalui baris kode `process.env.PORT || 3000`.

Hal yang Terjadi Jika Port Sudah Dipakai Aplikasi Lain

Server Gagal Berjalan (Crash): Node.js tidak bisa mengikat (bind) port yang sama secara bersamaan, sehingga server akan langsung berhenti dan memunculkan error `EADDRINUSE`

Solusinya yaitu Mengubah nilai PORT di dalam berkas `.env` ke nomor lain yang masih kosong (misalnya dari 3000 menjadi 4000) atau mematikan proses aplikasi lain yang sedang berjalan pada port tersebut (misalnya dengan perintah `taskkill /F /IM node.exe` pada terminal Windows).

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

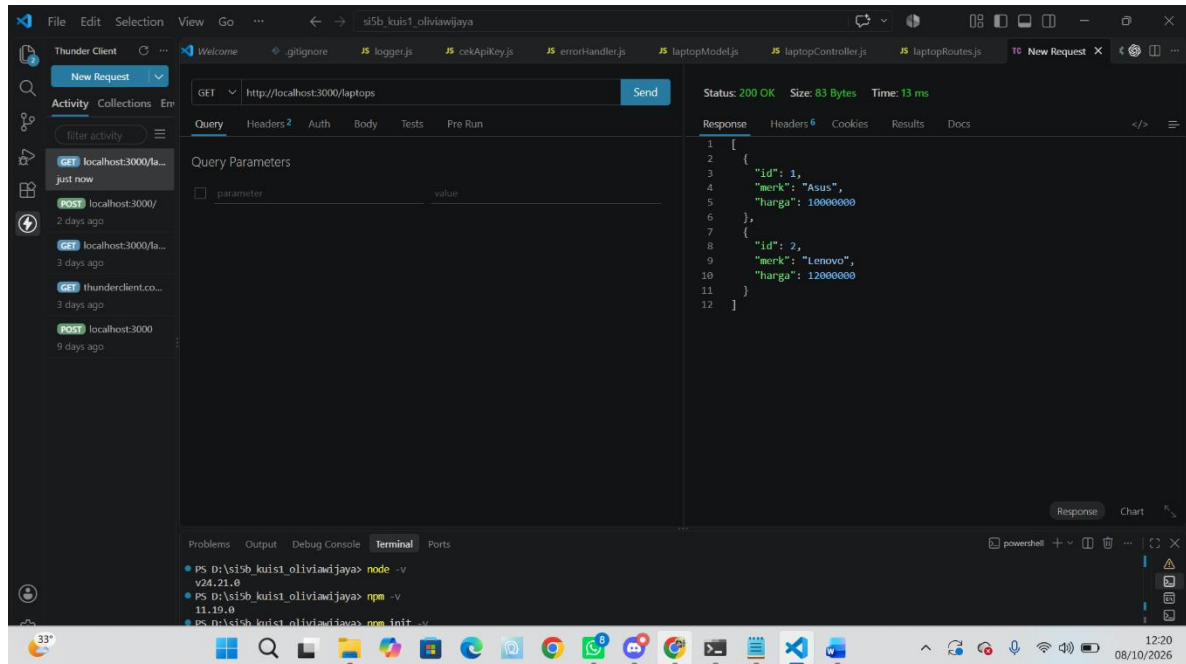
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /laptops)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /laptops
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK
Lulus?	[x] Ya [] Tidak



[Tempel screenshot di sini]

Gambar 4. Hasil GET semua data

Gambar 4. Hasil GET semua data

Penjelasan:

Jelaskan kode status yang muncul dan kemampuan endpoint ini. Sebutkan urutan pemanggilan: route mana, fungsi Controller mana, dan fungsi Model mana yang bekerja.

Kode status 200 OK menandakan bahwa permintaan HTTP GET berhasil diproses oleh server, di mana endpoint GET /laptops ini memiliki kemampuan untuk mengambil dan menampilkan seluruh koleksi data laptop dalam format array JSON.

Urutan Pemanggilan Backend:

Route (routes/laptopRoutes.js): Menerima request GET / pada path utama /laptops, lalu memetakan dan meneruskan pemanggilan ke fungsi controller.getAll.

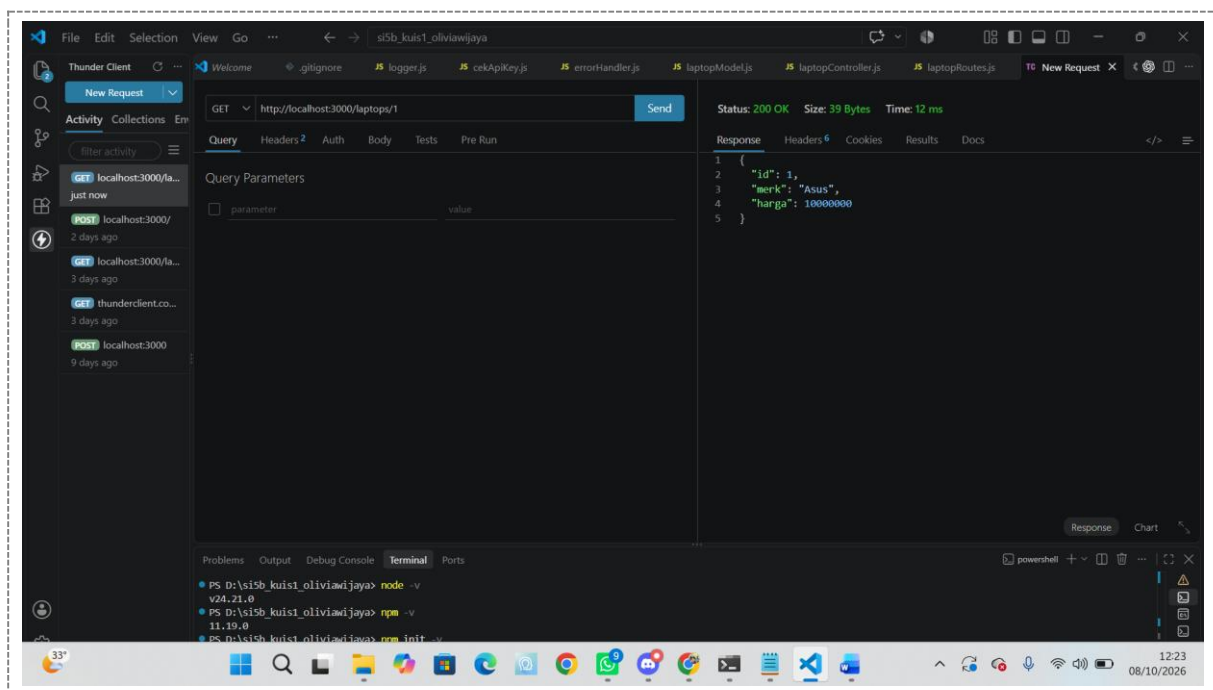
Controller (controllers/laptopController.js): Fungsi getAll memanggil fungsi Model.getAll(), menerima data dari model, lalu menyusun dan mengembalikan respons HTTP status 200 OK berisi data JSON ke klien.

Model (models/laptopModel.js): Fungsi getAll mengakses dan mengembalikan seluruh data dataLaptop dari memori untuk diberikan kembali kepada Controller.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /laptops/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK
Lulus?	[x] Ya



Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

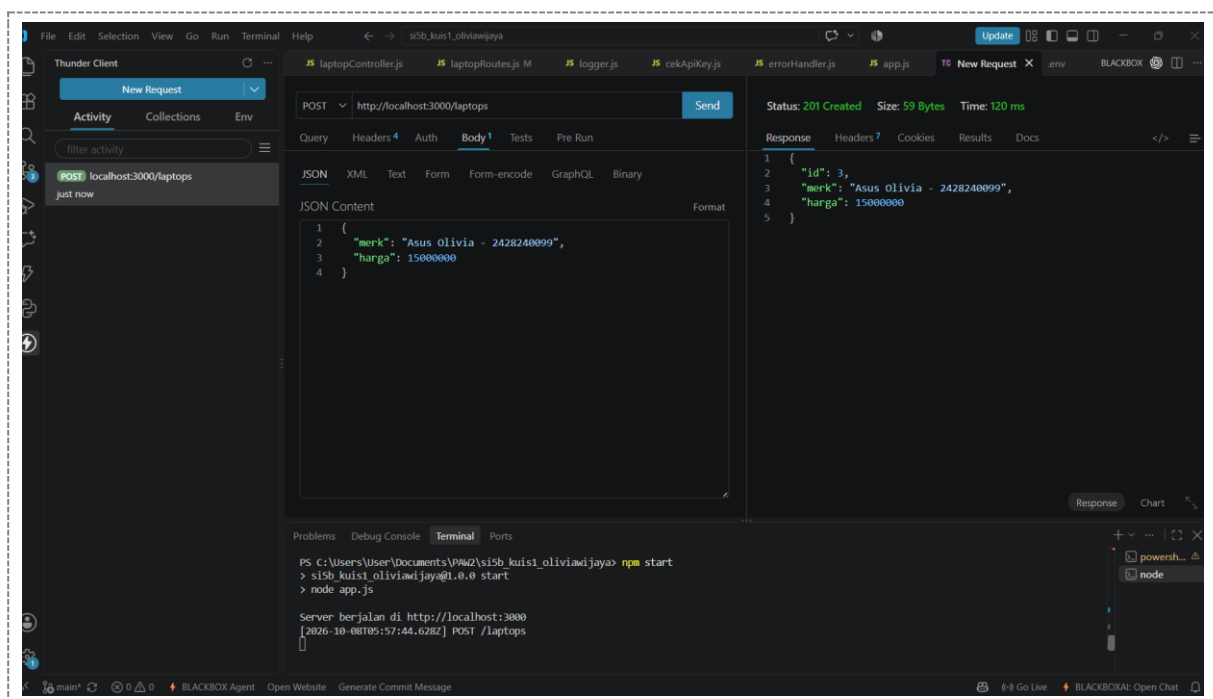
Jelaskan bagaimana ID dari alamat diambil (req.params) dan mengapa perlu diubah menjadi angka.

ID pada alamat URL ditangkap oleh Express.js melalui properti req.params.id sesuai dengan definisi rute /:id. Nilai yang didapat dari URL tersebut selalu bertipe teks (string), sehingga perlu diubah menjadi tipe angka (number) menggunakan fungsi parseInt() agar proses pencocokan ID pada model (item.id === parseInt(id)) dapat berjalan secara valid.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /[sumber-harga]
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "merk": "Asus olivia - 2428240099", "harga": 15000000 }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created
Lulus?	[x] Ya [] Tidak



Gambar 6. Hasil POST data baru

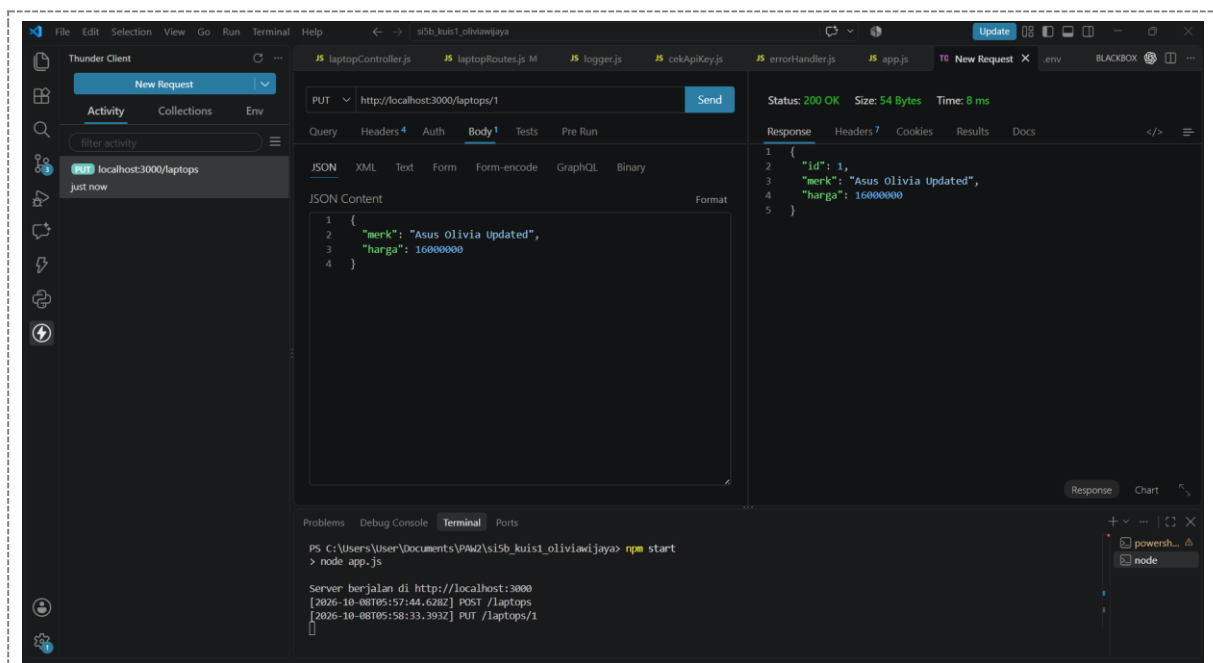
Penjelasan:

x-api-key diperiksa oleh middleware cekApiKey. Middleware express.json() bertugas mengurai (parse) body request yang berformat JSON menjadi objek JavaScript yang siap dibaca pada req.body. Status 201 Created digunakan (bukan 200) untuk menandakan secara spesifik bahwa sebuah sumber daya baru telah berhasil dibuat di server.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /laptops/1
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "merk": "Asus Olivia Updated", "harga": 16000000 }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK
Lulus?	[x] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

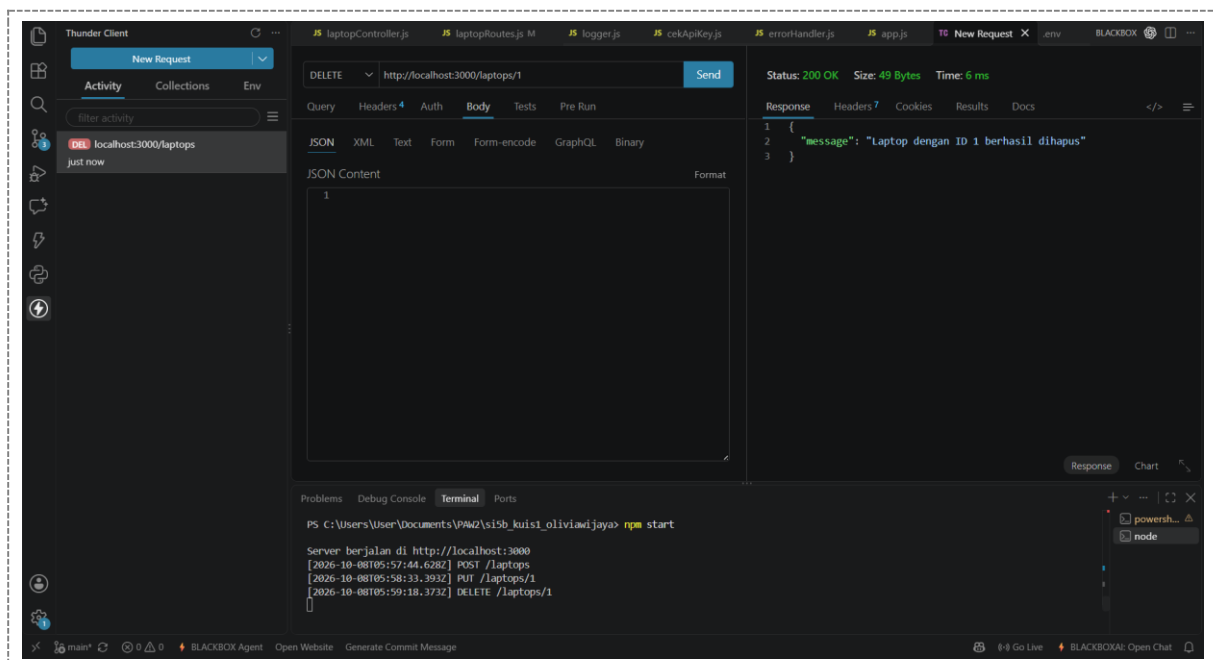
Penjelasan:

Field yang mengalami perubahan adalah merk (menjadi "Asus Olivia Updated") dan harga (menjadi 16000000), sedangkan field id tetap tidak berubah (nilainya 1). Perubahan data ini diproses pada layer **Model** (laptopModel.update) setelah mendapatkan kiriman data yang valid dari layer Controller.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /laptops/1
Header	x-api-key: kunci-rahasia-paw2
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	200 OK — {"message": "Laptop dengan ID 1 berhasil dihapus"}
Lulus?	☒ Ya ☐ Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Secara standar REST API, status **204 No Content** digunakan untuk mengindikasikan bahwa proses penghapusan berhasil dan server tidak perlu mengembalikan konten body apa pun. Namun pada implementasi ini, server mengembalikan status **200 OK** beserta pesan JSON konfirmasi bahwa data laptop dengan ID tersebut telah berhasil dihapus dari memori.

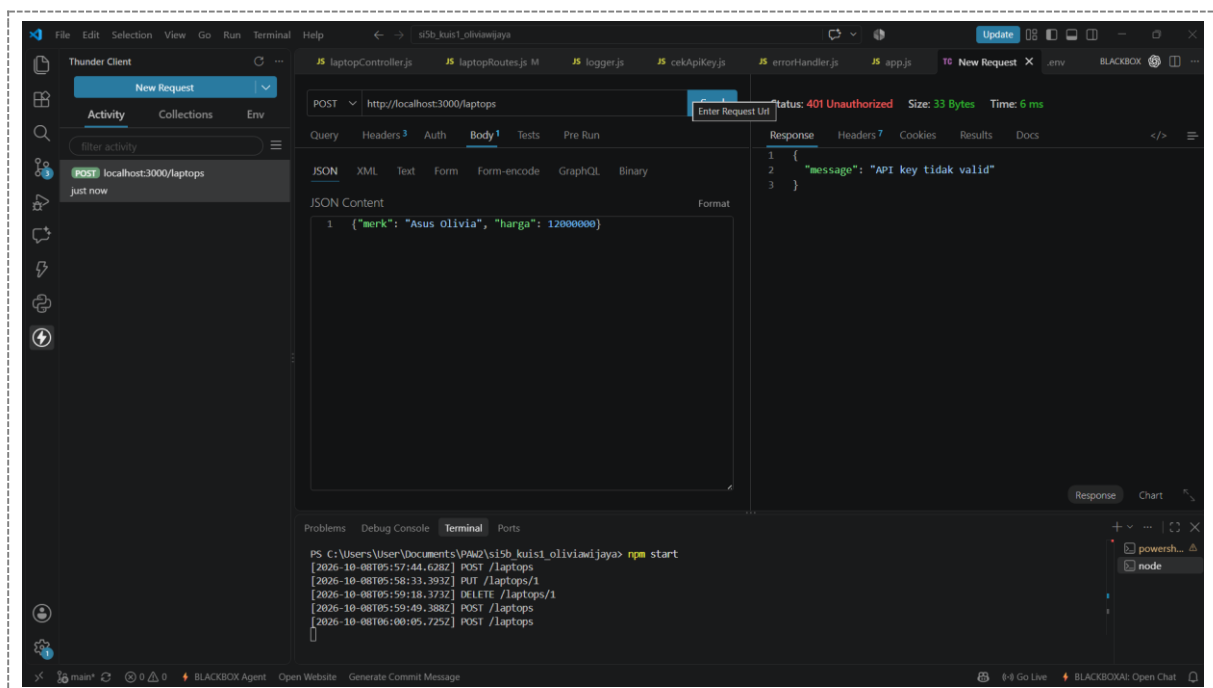
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /laptops (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ "merk": "Asus olivia", "harga": 12000000 }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	(401 Unauthorized — {"message": "API key tidak valid"})
Lulus?	[x] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

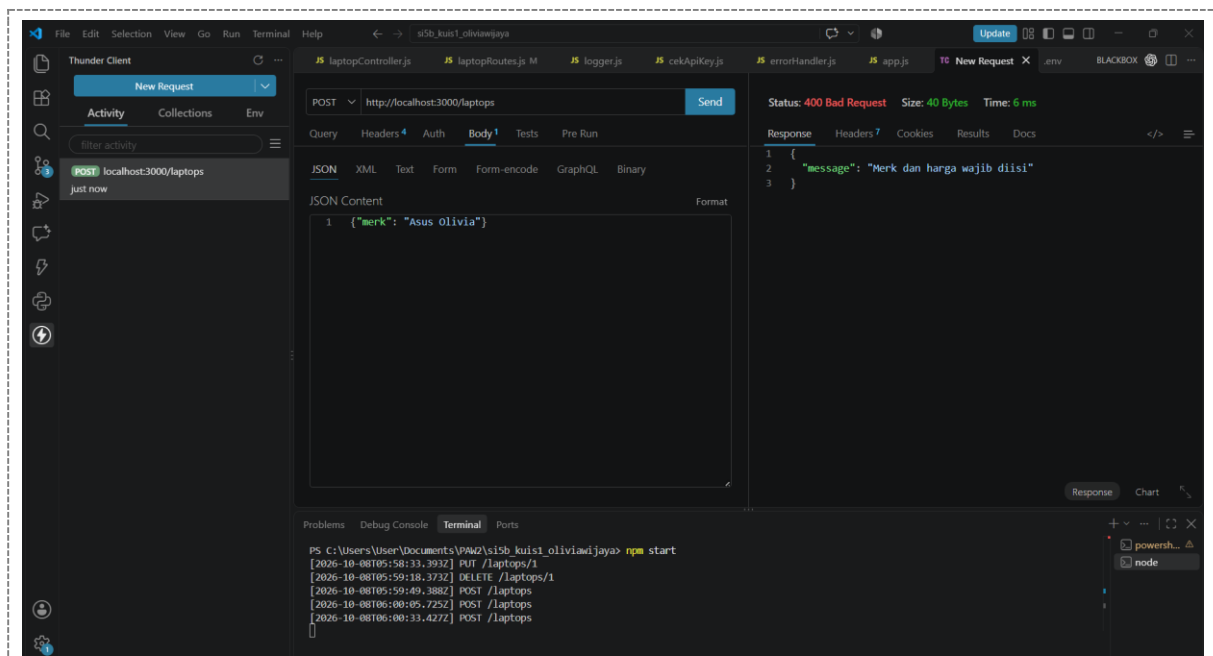
Penjelasan:

Permintaan ini ditolak oleh middleware `cekApiKey` karena tidak menemukan header `x-api-key` yang valid. Karena middleware langsung memutus alur eksekusi dan mengembalikan respons error 401, maka layer Controller tidak sempat dijalankan sama sekali.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /laptops
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "merk": "Asus olivia" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request — {"message": "Merk dan harga wajib diisi"}
Lulus?	[x] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

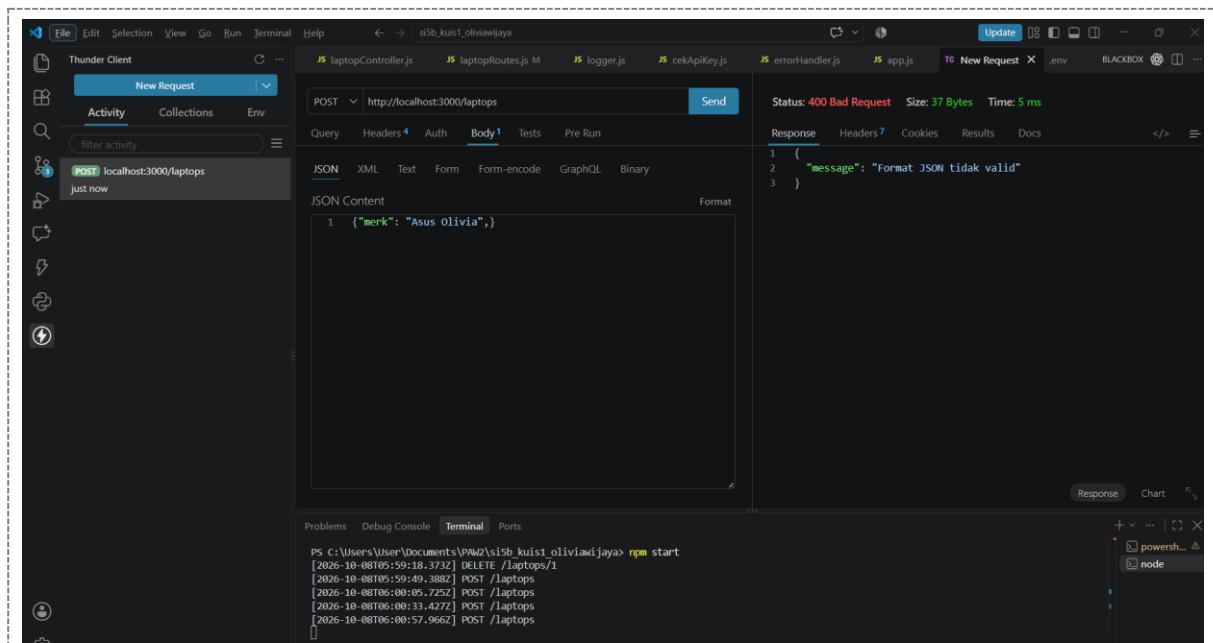
Penjelasan:

Validasi ini ditulis pada layer **Controller** (laptopController). Kode status yang digunakan adalah 400 Bad Request karena kesalahan terletak di sisi klien yang tidak melengkapi seluruh *field* wajib (merk dan harga) dalam struktur data yang dikirimkan.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (<i>request</i>)	POST /laptops
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "merk": "Asus olivia", }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request — {"message": "Format JSON tidak valid"}
Lulus?	[x] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

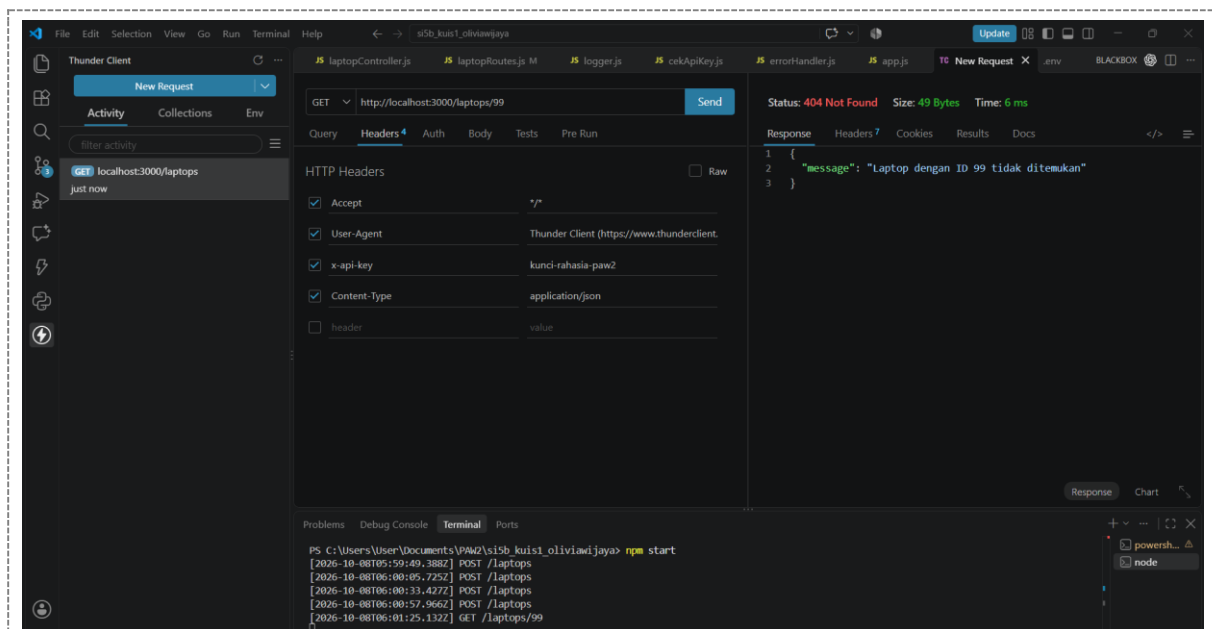
Penjelasan:

Format JSON yang rusak pertama kali dideteksi oleh middleware bawaan `express.json()`. Saat terjadi kesalahan parsing, error dilemparkan (*thrown*) dan ditangkap oleh middleware penanganan error terpusat (`errorHandler.js`), yang kemudian mengubah error sintaksis tersebut menjadi respons JSON berformat rapi dengan status 400 Bad Request.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /laptops/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found — {"message": "Laptop dengan ID 99 tidak ditemukan"}
Lulus?	[x] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Alur dimulai dari rute GET /laptops/99 yang memanggil fungsi `getById` pada **Controller**. Controller memanggil `laptopModel.getById(99)`. Karena ID 99 tidak ada dalam array data laptop, model mengembalikan `null/undefined`. Controller lalu mendeteksi hal tersebut dan langsung mengirimkan respons 404 Not Found dengan pesan JSON yang sesuai ke klien.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200 OK	YA
T02	Menampilkan satu data berdasarkan ID	200	200 OK	YA
T03	Menambah data baru	201	201 Created	YA
T04	Mengubah data	200	200 OK	YA
T05	Menghapus data	204	200 OK	YA
N01	Menambah data tanpa API key	401	401 Unauthorized	YA
N02	Menambah data dengan body tidak lengkap	400	400 Bad Request	YA
N03	Mengirim JSON yang rusak	400	400 Bad Request	YA
N04	Mengambil ID yang tidak ada	404	404 Not Found	YA

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	(https://github.com/oliviaolip/si5b_kuis1_oliviawija .git)
Pesan commit terakhir	refactor: structure backend architecture with MVC and middlewares
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

- **Route** bertugas mendefinisikan *endpoint* URL dan metode HTTP (misal: `router.get('/', laptopController.getAll)`).
- **Controller** berfokus mengolah logika bisnis serta mengatur *request* dan *response* HTTP (misal: `res.status(200).json(laptops)`).
- **Model** khusus mengurus manipulasi dan penyimpanan data aplikasi (misal: `const laptop = dataLaptop.find(l => l.id === id)`).

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Keuntungannya adalah membuat kode lebih rapi, konsisten, dan mudah dikelola (DRY — *Don't Repeat Yourself*), karena semua pengelolaan error standar atau tidak terduga ditangani di satu tempat tanpa perlu ditulis berulang di setiap rute. Buktinya terlihat pada pengujian **N03 (JSON Rusak)**, di mana error *parsing* JSON otomatis ditangkap oleh `errorHandler` terpusat dan menghasilkan respons JSON rapi tanpa membuat server terhenti atau membalas dalam format HTML.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	YA
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	YA
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	YA
4	Identitas saya (nama atau NIM) muncul pada data uji POST	YA
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	YA
6	Tabel rekapitulasi sesuai dengan tangkapan layar	YA
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	YA
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	YA

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100